Back to Reduction from previous lecture

- Things to note, assignment 2 posted

Reduction sum ends up moving the thread to the left most thread

```
for (unsigned int stride =1 ; stride <= blockDim.x; stride *=2){
  _syncthreads();
  if(t % stride == 0){
    partialSum[2*t] += partialSum[2*t + stride];
  }
}
```

For an example of 4 threads, we would want all threads to be active

- We start with a stride of 1 and keep halfing it each time, so we have 4 threads active, then 2 threads active, then 1 thread active
- Assignment will have something related to warp divergence, which is when threads in the same warp take different execution paths, leading to performance degradation.

– There will be inactive threads in a warp and fewer threads upon each iteration, which can lead to inefficient execution.

CUDA Pinned Memory & CUDA Streams

- CUDA pinned memory is a type of memory that is allocated on the host (CPU) and is page-locked, meaning it cannot be swapped out to disk. This allows for faster data transfer between the host and the device (GPU) because it can use direct memory access (DMA) instead of going through the CPU.
- CUDA streams are a sequence of operations that are executed in order on the GPU. They allow

for concurrent execution of multiple operations, which can improve performance by overlapping computation and data transfer. Each stream can execute independently, allowing for better utilization of the GPU resources.

CPU-GPU Data Transfer using DMA

- DMA (Direct Memory Access) allows for data transfer between the host and device without involving the CPU, which can significantly improve performance. When using pinned memory, the GPU can directly access the data in the host memory, reducing latency and increasing throughput.

– Used by cudaMemcpy() for better efficiency – Frees up CPU and prevents bogging down the system – Hardware unit specialized to transfer a number of bytes request by OS – Between physical memory address space regions – Uses system interconnect (PCIe) to transfer data between host and device

Virtual Memory Management

- Modern computers use virtual memory management

– Each virt addr spacve is divided into pages (4KB) that are mapped ito and out of the phys memory by the OS – Virt Mem pages can be paged out of phys memory to make room for other pages, and paged back in when needed – Whether or not a variable is in phys memory is checked at address translation timeo

Pinned Memory and DMA Data Transfer

- Pinned memory is page-locked, meaning it cannot be paged out to disk. This allows for faster data transfer between the host and device using DMA, as the GPU can directly access the data in the host memory without involving the CPU.

– They cannot be paged out – Allocated with special API (cudaMallocHost) and freed with cudaFreeHost – CPU memory that serves as the soruce or destination of a DMA transfer must be pinned memory

CUDA Data transfer uses pinned memory

- We never worked with memory in the lab, it was all abstracted by cudaMemcpy()
- DMA used by cudaMemcpy() requires any source or destination in the host memory is allocated as pinned memory.
- If Source or Destination of cudaMemcpy() is not pinned memory, it needs to be first copied to a pinned memory - extra overhead

Allocate/Free Pinned Memory

- cudaHostAlloc(), three parameters

1. Address of pointer to alloc memory
2. Size of alloc memory in bytes
3. Option - use `cudaHostAllocDefault` for default behavior, `cudaHostAllocPortable` to make it accessible by all CUDA contexts, `cudaHostAllocMapped` to map it into the device's address space, and `cudaHostAllocWriteCombined` for write-combined memory that is optimized for write operations.

- cudaFreeHost() to free pinned memory, one parameter

1. Pointer to pinned memory to free

Using Pinned Memory in CUDA

- Use allocated pinned memory and its pointer the same wayt as those returned by `malloc();`
- Only difference is that allocated memory is page-locked and cannot be paged out to disk, which allows for faster data transfer between host and device using DMA.

Putting it together - Vector Addition Host Code Example

```
int main(){
  float *h_A, *h_B, *h_C; // Host pointers
...

cudaHostAlloc((void**)&h_A, N* sizeof(float), cudaHostAllocDefault);
cudaHostAlloc((void**)&h_B, N* sizeof(float), cudaHostAllocDefault);
cudaHostAlloc((void**)&h_C, N* sizeof(float), cudaHostAllocDefault);
...
// cudaMemcpy() runs 2x faster with pinned memory

}
```

Serialized Data Transfer and Computation

- So far, the way we use `cudaMemcpy()` serializes data transfer and GPU computation for `VecAddAKernel()`

Device Overlap

- Some CUDA devices support device overlap

– Simulataneous execute a kernel while copying data between host and device

Ideal, Pipelined Timing

- Divide large vectors into segments
- Overlap transfer and compute of adjacent segments

– In the pipeline, we are copying in the data as we are computing the data. We are parallelizing the data transfer and computation, which can lead to significant performance improvements by reducing idle time for both the CPU and GPU.

CUDA Streams

- CUDA supports breaking things into concurrent streams of execution, which can be used to achieve device overlap.
- Each stream is a queue of operations that execute in order on the GPU, but different streams can execute concurrently. This allows for overlapping data transfer and computation, improving performance by maximizing GPU utilization.
- Operations (tasks) in different streams can go in parallel

– “Task parallelism” - different tasks in different streams can execute concurrently

Streams

- Requests are made from the host code are put in FIFO queues
- Queues are read and processed asynchronously by the driver and device
- To allow concurrent copy/execution, you require multiple streams
- CUDA “events” allow the host thread to query and synchronize with individual queues

Conceptializing View of Streams

- Stream 0 goes into copy engine
- Stream 1 goes into compute engine

Simple Multi-Stream Host Code Example

```
cudaStream_t stream0, stream1;
cudaStreamCreate(&stream0);
cudaStreamCreate(&stream1);
// Copy data for stream 0

for(int i = 0; i<n; i+= SegSize*2){
  cudaMemcpyAsync(d_A0 , h_A + i, SegSize*sizeof(float), cudaMemcpyHostToDevice,
stream0);
  cudaMemcpyAsync(d_B0 , h_B + i, SegSize*sizeof(float), cudaMemcpyHostToDevice,
stream0);
  VecAddAKernel<<<gridSize, blockSize, 0, stream0>>>(d_A0, d_B0, d_C0, SegSize);
  cudaMemcpyAsync(h_C + i, d_C + i, SegSize*sizeof(float), cudaMemcpyDeviceToHost,
stream0);

  // Copy data for stream 1
  cudaMemcpyAsync(d_A1 , h_A + i + SegSize, SegSize*sizeof(float),
cudaMemcpyHostToDevice, stream1);
  cudaMemcpyAsync(d_B1 , h_B + i + SegSize, SegSize*sizeof(float),
cudaMemcpyHostToDevice, stream1);
  VecAddAKernel<<<gridSize, blockSize, 0, stream1>>>(d_A1, d_B1, d_C1, SegSize);
  cudaMemcpyAsync(h_C + i + SegSize, d_C + i + SegSize, SegSize*sizeof(float),
cudaMemcpyDeviceToHost, stream1);
}
```

- We have two streams, not completely ideal because we have slight overlap, but we can have more streams to further improve performance by increasing the granularity of the tasks and allowing for more concurrent execution.
- We can reorder these asynchronous instructions for further optimization of pipeline.
- Have kernel 0 for copy the data and kernel 1 for computing the data, so we can have more overlap and better performance.

```
for (int i=0;i<n; i+=SegSize*2){
  // Copy data for stream 0
  cudaMemcpyAsync(d_A0 , h_A + i, SegSize*sizeof(float), cudaMemcpyHostToDevice,
stream0);
  cudaMemcpyAsync(d_B0 , h_B + i, SegSize*sizeof(float), cudaMemcpyHostToDevice,
stream0);

  // Copy data for stream 1
  cudaMemcpyAsync(d_A1 , h_A + i + SegSize, SegSize*sizeof(float),
cudaMemcpyHostToDevice, stream1);
  cudaMemcpyAsync(d_B1 , h_B + i + SegSize, SegSize*sizeof(float),
cudaMemcpyHostToDevice, stream1);

  // Compute for stream 0
  VecAddAKernel<<<gridSize, blockSize, 0, stream0>>>(d_A0, d_B0, d_C0, SegSize);
  cudaMemcpyAsync(h_C + i, d_C + i, SegSize*sizeof(float), cudaMemcpyDeviceToHost,
stream0);

  // Compute for stream 1
  VecAddAKernel<<<gridSize, blockSize, 0, stream1>>>(d_A1, d_B1, d_C1, SegSize);
  cudaMemcpyAsync(h_C + i + SegSize, d_C + i + SegSize, SegSize*sizeof(float),
cudaMemcpyDeviceToHost, stream1);
}
```

Now, the completely ideal Pipelined Timing

- We would need 3 buffers for our code

– After the first computation, we have stream0 with the C0 transfer, stream1 with C1 = A1 + B1, and stream 2 with A2 and B2 transfer.

– Wouldn't be possible with 2 streams becuase of the transfer of the previous result and the next transfer of the next data concurrently without another stream to handle the next transfer.

Wait until all tasks have completed

- `cudaStreamSynchronize(stream_id)` to wait for all tasks in all streams to complete for current device

– Takes one parameter - stream identifier – Wait all tasks in a stream to complete – `cudaStreamSynchronize(stream0)` to wait for all tasks in all streams to complete for current device

- `cudaDeviceSynchronize()` to wait for all tasks in all streams to complete

– Also use for host code – No parameter

Midterm on week5

- They will be quite conceptual
- What order is this code executed in?
- What's missing from this code?

– nothing too complicated, just basic understanding of how to use streams and pinned memory – Answers from reports on the assignment

- Seems like everything until today's lecture

– In class, week5 Tuesday